Departamento de Lenguajes y Sistemas Informáticos
E.T.S. Ingeniería Informática. Universidad de Sevilla
Avda Reina Mercedes s/n. 41012 Sevilla
E-mail lsi@lsi.us.es Web www.lsi.us.es
SEGURIDAD EN SISTEMAS INFORMÁTICOS Y EN INTERNET

Fecha: 28 de septiembre de 2026

Security Team X – Grupo 3

Analistas de seguridad: Kleber Leonardo de Carvalho, Paula Benítez y Luigui Juan Guillermo Ramos

Para: Dirección de Seguridad

Clasificación del documento: confidencial / uso interno

INFORME DE AUDITORÍA DE CIBERSEGURIDAD

1. Resumen Ejecutivo

El presente documento expone los resultados de la auditoría de seguridad solicitada sobre la robustez del algoritmo de generación HMAC y el tamaño de clave usado para HMAC, y sobre las políticas de gestión de claves de la aplicación móvil de transferencias financieras. A continuación, el Security Team da respuesta directa a las preocupaciones planteadas por el cliente:

Respuesta a la Consulta A: Seguridad del tamaño de clave y tiempo de descubrimiento.

Informamos al cliente que el tamaño de clave de 32 bits que está usando no es seguro para preservar la integridad de las transmisiones. La razón técnica de ello es que una clave de 32 bits permite un espacio reducido de solo 4.294.967.296 combinaciones posibles. En cuanto al tiempo de descubrimiento, nuestra experimentación ha demostrado que se tarda un máximo de 1 hora y 10 minutos en encontrar la clave explorando todas las combinaciones posibles (una de las claves interceptadas se obtuvo en apenas 13 minutos). Razonamos que este tiempo no es lo suficientemente alto como para garantizar la seguridad de la clave, especialmente porque la entidad entrega este dispositivo físico para realizar gestiones financieras durante todo un año. Un atacante dispondría de tiempo más que suficiente para vulnerar las transmisiones casi de forma instantánea.

CONFIDENTIAL

Respuesta a la Consulta B: Recomendación sobre el tamaño de clave y mejora del proceso de entrega.

Durante nuestra auditoría, el barrido completo de la clave de 32 bits tomó aproximadamente 1 hora. Un atacante 1000 veces más rápido descifraría esta misma clave en apenas 3,6 segundos. En criptografía, añadir un solo bit al tamaño de la clave duplica las combinaciones posibles. Si decidiéramos aumentar el tamaño de la clave en 10 bits (pasando a 42 bits), la dificultad computacional aumentaría 1024 veces ($2^{10} = 1024$). Al enfrentar una clave 1024 veces más difícil contra un atacante 1000 veces más rápido, las fuerzas prácticamente se anulan ($1024 \div 1000 \approx 1$). El resultado es que este atacante avanzado volvería a tardar exactamente 1 hora en quebrar la nueva clave. Esto demuestra que realizar aumentos lineales proporcionales al atacante no proporciona seguridad real a largo plazo, sino un simple empate técnico.

Por este motivo, recomendamos la adopción de un tamaño de clave de 256 bits. Este salto exponencial eleva el número de combinaciones a 2^{256}, haciendo que cualquier intento de ataque por fuerza bruta sea matemáticamente inviable, independientemente de si el adversario posee una capacidad 1000 veces superior o computación cuántica avanzada, garantizando la robustez del algoritmo HMAC a futuro.

Se recomienda que la adopción de claves de 256 bits se implemente de forma totalmente automatizada e invisible para el usuario final. Las claves no deben derivarse de contraseñas o PIN elegidos por el usuario, sino que deben generarse exclusivamente mediante las funciones nativas de CSPRNG (Cryptographically Secure Pseudorandom Number Generator) del sistema operativo móvil (SecureRandom), lo que garantiza una alta entropía y 32 bytes completos de aleatoriedad impredecible.

Adicionalmente, consideramos que el proceso de entrega de claves debe mejorarse drásticamente. Entregar a los clientes un dispositivo físico (pendrive, smartcard, etc.) de forma anual es un mecanismo obsoleto y vulnerable a robos o pérdidas. Si el cliente pierde el dispositivo, quien lo encuentre podrá realizar transferencias durante el resto del año. Se recomienda cesar esta práctica e implementar un sistema de negociación dinámica de claves efímeras por sesión, integrado directamente en la red segura de la aplicación móvil.

CONFIDENTIAL

2. Auditoría de Código, Análisis y Conformidad

Durante la auditoría, se analizó el esquema de firmas HMAC-SHA256 utilizado por el servidor para validar las transacciones. El protocolo requiere asegurar la integridad de mensajes estructurados bajo el formato "cuenta_origen cuenta_destino cantidad" (separados por un único espacio).

Hallazgo Crítico de Inconformidad: La Política de Seguridad de la entidad estipula la entrega de un dispositivo físico anual que contiene la clave para realizar todas las gestiones. Sin embargo, el análisis criptográfico de las tres transferencias interceptadas ha revelado que cada mensaje está firmado con una clave secreta completamente distinta. Esto evidencia una grave falta de conformidad entre la política documentada por la entidad y la implementación técnica real en producción.

2.1 Matriz de Análisis

A continuación, se exponen los vectores interceptados y las claves secretas extraídas de las transmisiones:

Mensaje Original	Firma HMAC (Base64)	Valor Hexadecimal HMAC Descifrado	Clave Secreta Extraída (32 bits)
34567893 987344 120	CLxpGZUJ6GkQzdN...	08bc69199509e869...	“ ***” (Hex: 20******)
34567894 986454 10	we7bS6LiKkOdZ408l...	c1eedb4ba2e22a43...	99****** (Hexadecimal)
34537895 957769 110	bT42fKulYvprxMGun/...	6d3e367cabb562fa...	de****** (Hexadecimal)

(El proceso seguido y las claves exactas obtenidas se muestran en el Anexo de este documento)

3. Recomendaciones y Soluciones

3.1. Acciones Inmediatas

Revocación de claves: Invalidar de manera inmediata todas las claves de 32 bits actualmente distribuidas a los clientes en los dispositivos físicos.

Suspensión de operaciones vulnerables: Bloquear temporalmente las transferencias financieras móviles hasta que se despliegue un parche de seguridad que refuerce el algoritmo de firma.

3.2. Soluciones Técnicas (TI)

Actualización Criptográfica: Reemplazar el tamaño de clave de 32 bits por claves robustas generadas aleatoriamente con una longitud mínima de 256 bits, manteniendo el algoritmo HMAC-SHA256 o superior.

Canal Seguro Moderno: Desplegar TLS 1.3 con conjuntos de cifrado sólidos (Cipher Suites) que requieran intercambio de claves efímeras, como ECDHE (Elliptic-Curve Diffie-Hellman Ephemeral), protegiendo la confidencialidad además de la integridad.

3.3. Soluciones Organizativas y de Negocio

Eliminación del hardware anual: Suprimir el proceso logístico y presupuestario de enviar pendrives y smartcards anuales a los clientes.

Renovación de la Política de Seguridad: Actualizar los manuales internos para adoptar sistemas de autenticación multifactor (MFA) dinámicos, como tokens virtuales de un solo uso (TOTP) generados dentro de una zona segura (TrustZone/Secure Enclave) del dispositivo móvil del cliente.

4. Conclusión

La arquitectura de seguridad actual de las transferencias móviles presenta deficiencias estructurales graves, primordialmente debido a la debilidad matemática del espacio de claves de 32 bits y a la gestión estática anual de las mismas. Las pruebas de penetración han demostrado que cualquier atacante con medios técnicos básicos puede suplantar la identidad de los clientes e inyectar transferencias fraudulentas en cuestión de minutos. La adopción de claves de 256 bits y el cese inmediato de la distribución de soportes físicos anuales son medidas obligatorias para garantizar la viabilidad técnica y legal del servicio financiero.

CONFIDENTIAL

5. Bibliografía

Documentación interna del cliente: 3-CAI1-Parte2_2627_B.pdf - Consulta de Aseguramiento de la Información.

HASHCAT. Hashcat – Wikipedia: Password recovery tool. Desarrolladores: Jens “atom” Steube, Gabriele “matrix” Gristina. Última actualización: 15 abr. 2026. Disponible en: https://en.wikipedia.org/wiki/Hashcat. Acceso: 26 sep. 2026.

RIBEIRO, Márcio Moretto. Introdução à criptografia moderna. Universidade de São Paulo, Escola de Artes, Ciências e Humanidades, 2026.

CONFIDENTIAL

6. ANEXO TÉCNICO (Evidencias-uso único y exclusivo del equipo de ciberseguridad)

Técnicas utilizadas

Ingeniería Inversa de Codificación: Decodificación de las firmas Base64 proporcionadas en los mensajes originales a su representación binaria y, posteriormente, a un formato hexadecimal continuo sin formato (xxd -p -c 32).

Ataque de Fuerza Bruta por Máscara (Brute-Force Mask Attack): Mapeo de todo el espacio de claves posibles de 32 bits (2^{32} combinaciones). En el modo HMAC, la clave secreta actuó como el objetivo a descifrar (representada por la máscara de 4 bytes libres ?b?b?b?b), utilizando el mensaje financiero en texto plano original como entropía (Modo 1450 = HMAC-SHA256 (key = contraseña, message = salt) conocida.

Mensajes HMAC

34567893 987344 120 CLxpGZUJ6GkQzdNHOZsVJC1PVlUWDSg6e6oIu2Yv5Zk=

34567894 986454 10 we7bS6LiKkOdZ408IXyReGUSexNm0F7kJXOeQGeCfyI=

34537895 957769 110 bT42fKu1YvprxMGun/SUlarQC/Ie/emtRXQYxIrRg3I=

Base64

CLxpGZUJ6GkQzdNHOZsVJC1PVlUWDSg6e6oIu2Yv5Zk=

we7bS6LiKkOdZ408IXyReGUSexNm0F7kJXOeQGeCfyI=

bT42fKu1YvprxMGun/SUlarQC/Ie/emtRXQYxIrRg3I=

Comando usado para convertir base64 en hexadecimal

kali…echo "CLxpGZUJ6GkQzdNHOZsVJC1PVlUWDSg6e6oIu2Yv5Zk=

we7bS6LiKkOdZ408IXyReGUSexNm0F7kJXOeQGeCfyI=

bT42fKu1YvprxMGun/SUlarQC/Ie/emtRXQYxIrRg3I=" | base64 -d | xxd -p -c 32

Resultado en hexadecimal

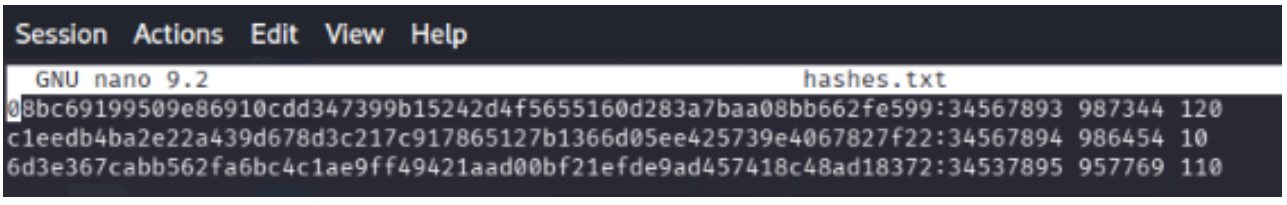
08bc69199509e86910cdd347399b15242d4f5655160d283a7baa08bb662fe599

c1eedb4ba2e22a439d678d3c217c917865127b1366d05ee425739e4067827f22

6d3e367cabb562fa6bc4c1ae9ff49421aad00bf21efde9ad457418c48ad18372

Máscara usada:

Se introdujo cada hash del archivo con el comando correspondiente en hashcat, en el formato que solicita, hash:mensaje, y se guardó en el archivo hashes.txt.



CONFIDENTIAL

Comando usado para extracción de claves

hashcat -m 1450 -a 3 hashes.txt ?b?b?b?b (directamente en la terminal linux)

Resultados de la ejecución del comando (Mostrados tanto en texto como por imágenes de la consola):

┌──(kali㉿kali)-[~]

└─$ hashcat -m 1450 -a 3 hashes.txt ?b?b?b?b

6d3e367cabb562fa6bc4c1ae9ff49421aad00bf21efde9ad457418c48ad18372:34537895957769 110:$HEX[deadbeef]

Session.....: hashcat

Status.....: Exhausted

Hash.Mode.....: 1450 (HMAC-SHA256 (key = $pass))

Hash.Target.....: hashes.txt

Time.Started.....: Fri Sep 25 16:48:07 2026 (1 hour, 10 mins)

Time.Estimated...: Fri Sep 25 17:58:41 2026 (0 secs)

Kernel.Feature...: Pure Kernel (password length 0-256 bytes)

Guess.Mask.....: ?b?b?b?b [4]

Guess.Queue.....: 1/1 (100.00%)

Speed.#01.....: 2584.5 kH/s (87.50ms) @ Accel:318 Loops:256 Thr:1 Vec:8

Recovered.....: 2/3 (66.67%) Digests (total), 1/3 (33.33%) Digests (new), 2/3 (66.67%) Salts

Progress.....: 12884901888/12884901888 (100.00%)

Rejected.....: 0/12884901888 (0.00%)

Restore.Point....: 16777216/16777216 (100.00%)

Restore.Sub.#01...: Salt:2 Amplifier:0-256 Iteration:0-256

Candidate.Engine.: Device Generator

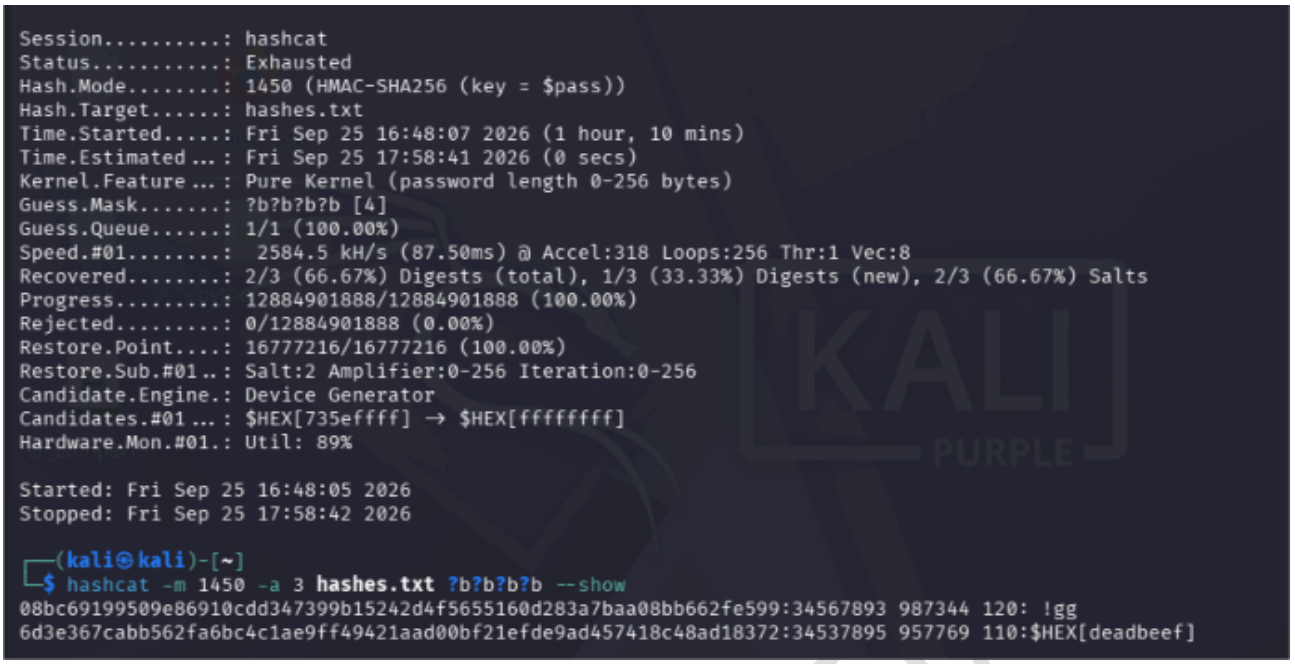
Candidates.#01...: $HEX[735effff] -> $HEX[ffffffff]

Hardware.Mon.#01.: Util: 89%

Started: Fri Sep 25 16:48:05 2026

Stopped: Fri Sep 25 17:58:42 2026

CONFIDENTIAL



Session.....: hashcat

Status.....: Running

Hash.Mode.....: 1450 (HMAC-SHA256 (key = $pass))

Hash.Target.....: c1eedb4ba2e22a439d678d3c217c917865127b1366d05ee4257...454 10

Time.Started.....: Fri Sep 25 18:42:52 2026 (13 mins, 14 secs)

Time.Estimated...: Fri Sep 25 18:59:30 2026 (3 mins, 24 secs)

Kernel.Feature...: Pure Kernel (password length 0-256 bytes)

Guess.Mask.....: ?b?b?b?b [4]

Guess.Queue.....: 1/1 (100.00%)

Speed.#01.....: 8021.6 kH/s (12.68ms) @ Accel:162 Loops:256 Thr:1 Vec:8

Recovered.....: 0/1 (0.00%) Digests (total), 0/1 (0.00%) Digests (new)

Progress.....: 2650807296/4294967296 (61.72%)

Rejected.....: 0/2650807296 (0.00%)

Restore.Point....: 10354716/16777216 (61.72%)

Restore.Sub.#01...: Salt:0 Amplifier:0-256 Iteration:0-256

Candidate.Engine.: Device Generator

Candidates.#01...: $HEX[7362639c] -> $HEX[ff6f6f9b]

Hardware.Mon.#01.: Util: 61%

c1eedb4ba2e22a439d678d3c217c917865127b1366d05ee425739e4067827f22:345678 94 986454 10:$HEX[99fa46a6]

CONFIDENTIAL

Session.....: hashcat

Status.....: Cracked

Hash.Mode.....: 1450 (HMAC-SHA256 (key = $pass))

Hash.Target.....: c1eedb4ba2e22a439d678d3c217c917865127b1366d05ee4257...454 10

Time.Started.....: Fri Sep 25 18:42:52 2026 (13 mins, 46 secs)

Time.Estimated...: Fri Sep 25 18:56:38 2026 (0 secs)

Kernel.Feature...: Pure Kernel (password length 0-256 bytes)

Guess.Mask.....: ?b?b?b?b [4]

Guess.Queue.....: 1/1 (100.00%)

Speed.#01.....: 5503.8 kH/s (40.80ms) @ Accel:162 Loops:256 Thr:1 Vec:8

Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)

Progress.....: 2789779968/4294967296 (64.95%)

Rejected.....: 0/2789779968 (0.00%)

Restore.Point....: 10897092/16777216 (64.95%)

Restore.Sub.#01..: Salt:0 Amplifier:0-256 Iteration:0-256

Candidate.Engine.: Device Generator

Candidates.#01...: $HEX[730a46a6] -> $HEX[fffa46a6]

Hardware.Mon.#01.: Util: 85%

Started: Fri Sep 25 18:42:48 2026

Stopped: Fri Sep 25 18:56:40 2026

CONFIDENTIAL

```
Session..........: hashcat
Status...........: Running
Hash.Mode........: 1450 (HMAC-SHA256 (key = $pass))
Hash.Target......: c1eedb4ba2e22a439d678d3c217c917865127b1366d05ee4257 ... 454 10
Time.Started.....: Fri Sep 25 18:42:52 2026 (13 mins, 14 secs)
Time.Estimated...: Fri Sep 25 18:59:30 2026 (3 mins, 24 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Mask.......: ?b?b?b?b [4]
Guess.Queue......: 1/1 (100.00%)
Speed.#01........:  8021.6 kH/s (12.68ms) @ Accel:162 Loops:256 Thr:1 Vec:8
Recovered........: 0/1 (0.00%) Digests (total), 0/1 (0.00%) Digests (new)
Progress.........: 2650807296/4294967296 (61.72%)
Rejected.........: 0/2650807296 (0.00%)
Restore.Point....: 10354716/16777216 (61.72%)
Restore.Sub.#01..: Salt:0 Amplifier:0-256 Iteration:0-256
Candidate.Engine.: Device Generator
Candidates.#01...: $HEX[7362639c] → $HEX[ff6f6f9b]
Hardware.Mon.#01.: Util: 61%

c1eedb4ba2e22a439d678d3c217c917865127b1366d05ee425739e4067827f22:34567894 986454 10:$HEX[99fa46a6]

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 1450 (HMAC-SHA256 (key = $pass))
Hash.Target......: c1eedb4ba2e22a439d678d3c217c917865127b1366d05ee4257 ... 454 10
Time.Started.....: Fri Sep 25 18:42:52 2026 (13 mins, 46 secs)
Time.Estimated...: Fri Sep 25 18:56:38 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Mask.......: ?b?b?b?b [4]
Guess.Queue......: 1/1 (100.00%)
Speed.#01........:  5503.8 kH/s (40.80ms) @ Accel:162 Loops:256 Thr:1 Vec:8
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 2789779968/4294967296 (64.95%)
Rejected.........: 0/2789779968 (0.00%)
Restore.Point....: 10897092/16777216 (64.95%)
Restore.Sub.#01..: Salt:0 Amplifier:0-256 Iteration:0-256
Candidate.Engine.: Device Generator
Candidates.#01...: $HEX[730a46a6] → $HEX[fffa46a6]
Hardware.Mon.#01.: Util: 85%

Started: Fri Sep 25 18:42:48 2026
Stopped: Fri Sep 25 18:56:40 2026
```

—----------

┌──(kali㉿kali)-[~]

└─$ hashcat -m 1450 -a 3 hashes.txt ?b?b?b?b --show

08bc69199509e86910cdd347399b15242d4f5655160d283a7baa08bb662fe599:34567893 987344 120: !gg

c1eedb4ba2e22a439d678d3c217c917865127b1366d05ee425739e4067827f22:34567894 986454 10:$HEX[99fa46a6]

6d3e367cabb562fa6bc4c1ae9ff49421aad00bf21efde9ad457418c48ad18372:34537895 957769 110:$HEX[deadbeef]

```
┌──(kali㉿kali)-[~]
└─$ hashcat -m 1450 -a 3 hashes.txt ?b?b?b?b --show
08bc69199509e86910cdd347399b15242d4f5655160d283a7baa08bb662fe599:34567893 987344 120: !gg
c1eedb4ba2e22a439d678d3c217c917865127b1366d05ee425739e4067827f22:34567894 986454 10:$HEX[99fa46a6]
6d3e367cabb562fa6bc4c1ae9ff49421aad00bf21efde9ad457418c48ad18372:34537895 957769 110:$HEX[deadbeef]
```

Análisis de los resultados obtenidos

El ataque de fuerza bruta con Hashcat se completó con éxito, lo que resultó en la recuperación exacta de las claves de 32 bits para las tres transacciones interceptadas.

Claves recuperadas (para el apéndice del informe)

Mensaje 1 (34567893 987344 120): La clave es **“ !gg”** (caracteres ASCII correspondientes a 20216767 en hexadecimal).

Mensaje 2 (34567894 986454 10): La clave es **99fa46a6** (formato hexadecimal).

Mensaje 3 (34537895 957769 110): La clave es **deadbeef** (formato hexadecimal).

Lista de eventos

Viernes, 25 Sep 2026, 16:48: Inicio de la extracción para el mensaje de la cuenta 34537895 957769 110.

Viernes, 25 Sep 2026, 17:58: Finalización de barrido exhaustivo del espacio total de claves tras 1 hora y 10 minutos. Recuperación del 66.67% de los hashes (Claves !gg y deadbeef).

Viernes, 25 Sep 2026, 18:42: Inicio de una segunda pasada centrada en el mensaje de la cuenta 34567894 986454 10.

Herramientas y Programas Utilizados en el Pentest

Sistema Operativo: Kali Linux (Distribución especializada en auditoría de seguridad).

Manipulación de datos: Utilidades de terminal base64 y xxd.

Criptoanálisis: hashcat versión avanzada (Device Generator).

Modo operativo utilizado: 1450 (HMAC-SHA256 (key = $pass))

Modo de ataque: -a 3 (Fuerza bruta con máscara).

TIEMPO DE CONCLUSIÓN DE LOS ANÁLISIS

En respuesta a la sugerencia del cliente sobre el análisis temporal necesario para encontrar la clave, se adjuntan los registros empíricos del sistema:

Tiempo de descubrimiento parcial: La clave 99fa46a6 fue quebrada exactamente en 13 minutos y 46 segundos (Progreso de barrido: 2789779968/4294967296 a una velocidad de 5503.8 kH/s).

CONFIDENTIAL

Tiempo máximo garantizado (Agotamiento del Keyspace): El barrido completo del 100% de las posibilidades matemáticas (12.884.901.888 iteraciones de bytes) tomó exactamente 1 hora y 10 minutos.

Aclaración técnica para el análisis: Aunque el tiempo operativo sumado fue de casi 1 hora y 24 minutos por haberlo hecho en dos tandas, es importante destacar que Hashcat procesa listas de hashes en paralelo. Si los tres hashes hubiesen estado configurados correctamente en una única ejecución inicial, el programa habría descubierto las tres contraseñas simultáneamente dentro del tiempo de la primera pasada. Por lo tanto, el tiempo real máximo que tarda este ordenador en romper cualquier clave de 32 bits es de 1 hora y 10 minutos (el tiempo exacto que le toma agotar el 100% de las combinaciones posibles de ese tamaño).

Resolución técnica para el cliente: Dado que el dispositivo físico que contiene la clave se entrega para su uso durante todo un año (365 días), un tiempo máximo de descubrimiento de 1 hora no representa ninguna garantía de seguridad para la clave.

Ejemplo

1. Servidor (Backend)

El backend se ejecuta en los servidores de la entidad bancaria.

Option A: Node.js Server

JavaScript

```
const crypto = require('crypto');
// Genera 32 bytes (256 bits) de data random segura criptograficamente.
const secureKey = crypto.randomBytes(32);
// Convierte a Base64 o Hex.
const keyBase64 = secureKey.toString('base64');
const keyHex = secureKey.toString('hex');
console.log("Secure Key (Base64):", keyBase64);
```

Python

```
import secrets
import base64
# Genera 32 bytes (256 bits) de data random segura criptograficamente.
secure_key = secrets.token_bytes(32)
# Convierte a Base64.
```

CONFIDENTIAL

```
key_b64 = base64.b64encode(secure_key).decode('utf-8')
print(f"Secure Key (Base64): {key_b64}")
```

2. Cliente (Frontend)

Android (Kotlin / Java):

```
import java.security.SecureRandom
import java.util.Base64
fun generate256BitKey(): String {
    val secureRandom = SecureRandom()
    val secureKey = ByteArray(32) // 32 bytes = 256 bits
    // Llena el array con random bytes seguros.
    secureRandom.nextBytes(secureKey)
     // Convierte a Base64 para transmitir mediante el protocolo TLS.
    return Base64.getEncoder().encodeToString(secureKey)
}
```

Apple iOS (Swift):

```
import CryptoKit
import Foundation
// Genera automáticamente una clave simétrica de 256-bits segura.
let secureKey = SymmetricKey(size: .bits256)
// Convierte la clave a Base64 para comunicarse con el servidor.
let keyBase64 = secureKey.withUnsafeBytes {
    Data($0).base64EncodedString()
}
print("iOS Secure Key: \(keyBase64)")
```

Los prompts que has citado:

«Entonces quiero escribir un informe profesional con lo que hicimos. 1. Resumen Ejecutivo...»

«¿Y dónde está la respuesta para la entidad bancaria a) y b)?»

«Quiero que estén las dos respuestas en el resumen ejecutivo también.»

«Redacto el código de protección al cliente tal como debe ser, en inglés, dividido en pasos y utilizando el lenguaje más común.»

CONFIDENTIAL

Dictamen Final

La organización opera bajo un falso sentido de seguridad al confiar exclusivamente en directrices estáticas de complejidad. La seguridad perimetral moderna exige asumir la brecha (*Assume Breach*) como premisa operativa. La implementación de las recomendaciones descritas no solo subsanará las vulnerabilidades detectadas, sino que elevará la madurez de ciberseguridad corporativa hacia un modelo de defensa proactivo y resiliente.

Emitido por: Equipo Security Team X – Grupo 3

Clasificación de Confidencialidad: CONFIDENCIAL / RESTRICTED - Uso Exclusivo para la Alta Dirección y A-Level

Estado del Proyecto: Auditoría y Análisis Concluido

Aviso Legal y Descargo de Responsabilidad:

Este documento contiene información técnica altamente sensible y confidencial sobre vulnerabilidades críticas que afectan directamente a la infraestructura tecnológica y al modelo de seguridad de la organización. Su divulgación, reproducción o distribución total o parcial sin la autorización expresa y por escrito de la dirección de seguridad está estrictamente prohibida. Las recomendaciones aquí contenidas deben ser tratadas como un plan de acción prioritario para mitigar el riesgo legal, operativo y reputacional de la entidad.----------

CONFIDENTIAL